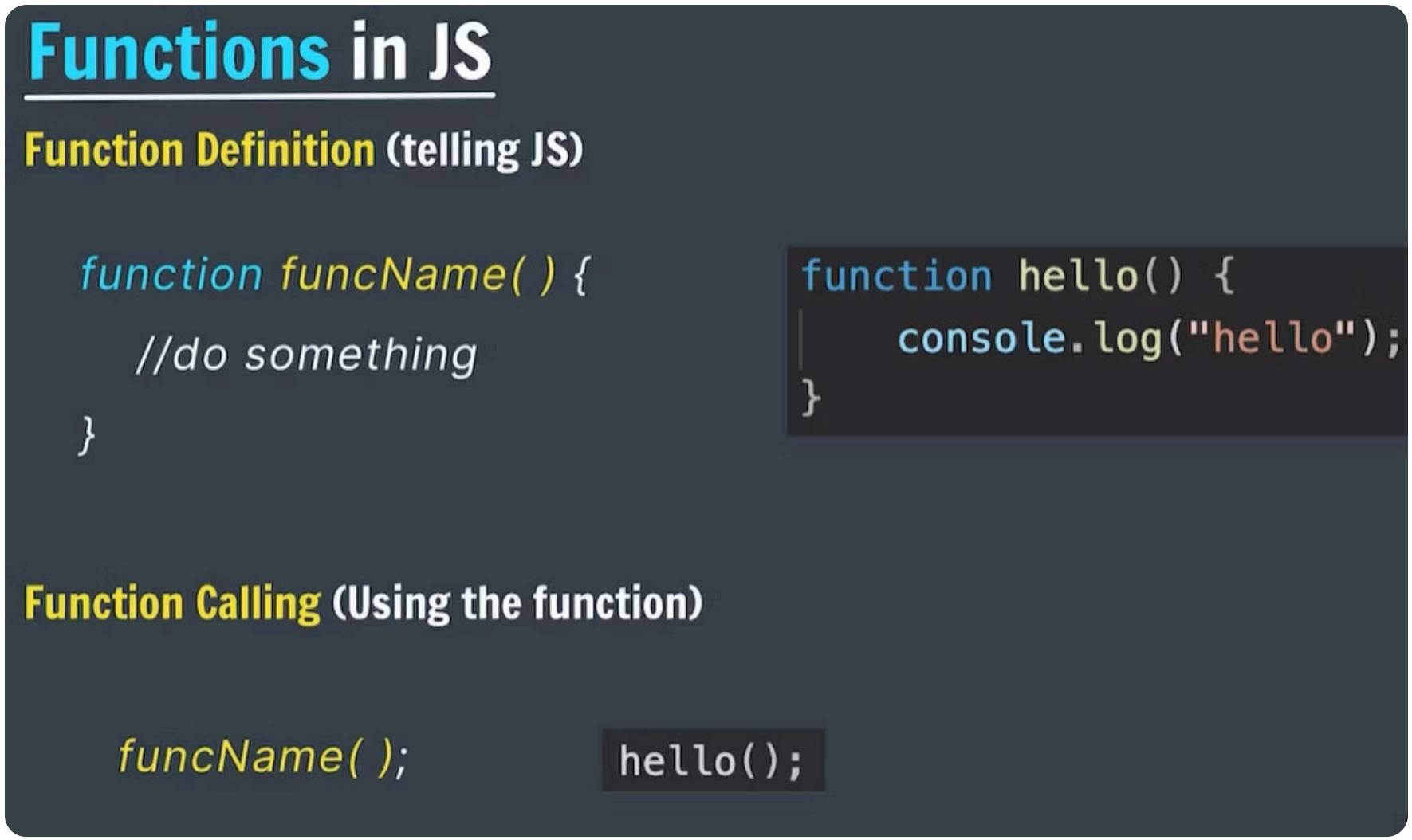
What is a Function?

In JavaScript, a function is a fundamental building block that encapsulates a specific task. It's a powerful tool for code organization and efficiency.

- A reusable block of code designed to perform a particular task.
- Functions are executed when they are **"called"** (invoked).
- They help organize code, reduce repetition, and improve readability.
- Think of them as "mini-programs" inside your main application.

```
function greet() {
  console.log("Hello, World!");
}

// Invoking the function
greet(); // Output: Hello, World!
```



Practice Qs

Create a Function to roll a dice & always display the value of the dice (1 to 6).

Ways to Define Functions

Functions with Arguments

Values we pass to the function

```
function funcName(arg1, arg2.. ) {
    //do something
}
```

```
function printInfo(name, age) {
    console.log(`${name}'s age is ${age}.`);
}
```

Practice Qs

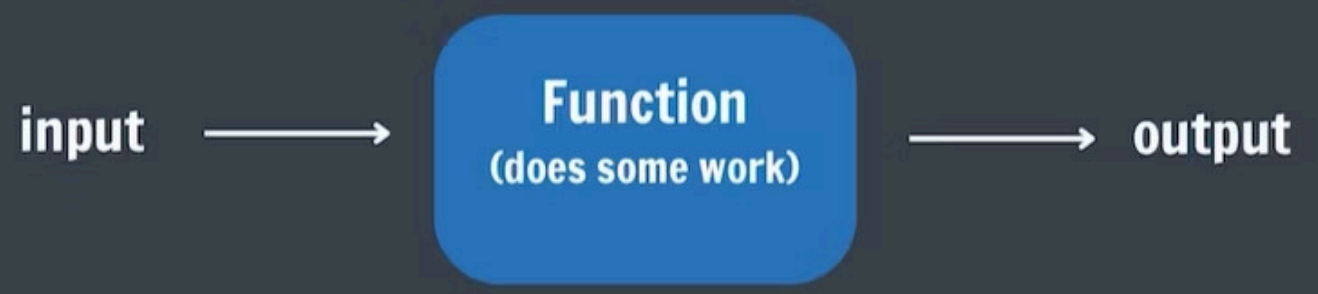
Create a Function that gives us the average of 3 numbers.

Practice Qs

Create a Function that prints the multiplication table of a number.

Return

return keyword is used to return some value from the function.




```
function funcName(arg1, arg2.. ) {
    //do something
    return val;
}
```

```
function sum(a, b) {
    return a+b;
}
```

Practice Qs

Create a Function that returns the sum of numbers from 1 to n.

Practice Qs

Create a Function that returns the concatenation of all strings in an array.

```
let str = ["hi", "hello", "bye", "!"];
```

Scope

Scope determines the accessibility of variables, objects, and functions from different parts of the code.

- Function
- Block
- Lexical

Function Scope

Variables defined inside a function are not accessible (visible) from outside the function.

```
function calSum(a, b) {
    let sum = a+b;
}

console.log(sum);
```

Block Scope

Variables declared inside a { } block cannot be accessed from outside the block.

```
for(let i=1; i<=5; i++) {
    console.log(i);
}

console.log(i);
```

```
let age = 25;
if(age >= 18) {
    let str = "adult";
}

console.log(str);
```

Lexical Scope

A function can use:

1. its **own local variables**, and
2. variables from **outer scopes that surround it in the code** (parent → grandparent → global).

```
let globalVar = "G";

function outer() {
  let outerVar = "O";

  function inner() {
    let innerVar = "I";
    console.log(globalVar, outerVar, innerVar); // ✅ G O I
  }

  inner();
}

outer();
```

Practice Qs

What will be the output?

```
let greet = "hello";

function changeGreet() {
    let greet = "namaste";
    console.log(greet);
    function innerGreen() {
        console.log(greet);
    }
}

console.log(greet);
changeGreet();
```

Function Expressions

```
const variable = function(arg1, arg2..) {
    //do or return something
}
```

```
const sum = function(a, b) {
    return a + b;
}

sum(2, 3);
```

Higher Order Functions

A function that does one or both :

- takes one or multiple functions as arguments
- returns a function

Higher Order Functions

Takes one or multiple functions as arguments

```
function multipleGreet(func, n) {
    for(let i=1; i<=n; i++) {
        func();
    }
}

let greet = function() {
    console.log("hello");
}

multipleGreet(greet, 2);
```

```
multipleGreet(function() {console.log("namaste")}, 1000);
```

Returns a function

```
function oddEvenTest(request) {
    if(request == "odd") {
        return function(n) {
            console.log(!(n%2 == 0));
        }
    } else if(request == "even") {
        return function(n) {
            console.log(n%2 == 0);
        }
    } else {
        console.log("wrong request");
    }
}
```

Methods

Actions that can be performed on an object.

```
const calculator = {
    add: function(a, b) {
        return a + b;
    },
    sub: function(a, b) {
        return a - b;
    },
    mul: function(a, b) {
        return a * b;
    }
};
```

```
> [1, 2, 3]
<: ▶ (3) [1, 2, 3]
> typeof [1, 2, 3]
<: 'object'
> [1, 2, 3].push(4);
<: 4
> [1, 2, 3].pop();
```